# Day 19 — Log Levels, Data Formats & Converters

## Part A — Log Levels

## 1. Where to Set the Log Level

**Monitor tab → Manage Integration Content → select the deployed artifact (iFlow) → Artifact Details → Log Configuration** — there you pick the log level for that iFlow.

**Log levels listed in class:** ① None ② Info ③ Error ④ Debug ⑤ Trace. We need to understand **why** each level exists, i.e. when to use which.

> **Demo setup used in class:** Postman → HTTPS adapter → **2 Content Modifiers** (changing the message body data) → so each log level can be compared on the same simple flow.

## 2. The Levels, One by One

### ① Info — the default

**By default, when you deploy the iFlow the log level will be Info.** Info tells you **what happened to that particular message — successful or failed** — together with its **MsgID**. **For every message a unique ID is generated.**

### ② None

**No MPL (Message Processing Logs) are created** — even if the message is successful or fails, for whatever reason, **no logs will be created for the message.**

> **Scenario:** if you don't want to create any MPL for an iFlow, you can use this option.

> **Caution:** with None, support cannot see *anything* in Monitor — not even failures. Use it only for very high-volume, non-critical flows, never for flows someone has to support.

### ③ Error

If the iFlow is triggered and message processing **fails for some reason**, and we have chosen the level **Error**, then **the error details will be logged**. (Successful messages are not logged in detail — this keeps log volume small.)

### ④ Trace — "the main concept of log level"

**What is the use of Trace?** Whenever you use Trace, **you can see the message content at each and every step.**

**Example:** source connected over HTTPS → message content, **headers, exchange properties, payload — everything will be captured.**

> **This option stays enabled for only 10 minutes, and you can see that traced log for 1 hour.**

> **Verified (same as Day 8):** SAP documents Trace as expiring after 10 minutes by default (the level then switches back to what it was before), with trace data cleared from the tenant after 60 minutes. While Trace is active, the MPL is written at the highest detail (like Debug) *plus* the message content.

### ⑤ Debug

If I want to **debug the message flow — which flow path the message is moving through** — especially in **complex scenarios with multiple directs / multiple receivers**.

- **Debug stays enabled for 24 hours**, and after that it is reset.

> **Clarification:** your notes say "after that Debug will get deleted". More precisely, after 24 hours the **log level automatically reverts** (normally to Info; one SAP Knowledge Base article notes some tenants revert to Error instead). The logs already written are not deleted — they follow the normal retention (section 3).
>
> **Also important: Debug shows every step executed, but NOT the message content** (no payload, headers or properties).

> Only **Trace** shows content. That is the key difference between the two.

### Comparison Table

| Level | What Is Logged | Payload / Headers / Properties? | Expires? | Typical Use |
|---|---|---|---|---|
| **None** | Nothing | No | — | Very high-volume, non-critical flows |
| **Error** | Only failed messages, with error details | No | — | Keep logs small; only failures matter |
| **Info** | Basic info per message (status, MsgID, timing); more detail on the last steps of failed messages | No | — | **Default; SAP recommends it for production** |
| **Debug** | Each step executed in the flow | No | 24 hours | Understanding the path in complex flows |
| **Trace** | Debug-level detail **plus message content** at every step | **Yes** | 10 min (content kept 1 hour) | Investigating data problems — dev/test only |

**Added facts (verified):**

- SAP's classic documentation lists four levels (None, Info, Debug, Trace); the **Error** option appears in newer tenant screens, as in your notes.
- **Trace and Debug should be used only in development/test** — payloads can contain sensitive data and these levels add load.
- The log level is **retained when you redeploy**, but if you **undeploy and deploy** again it resets to **Info**.
- You can override the level for a **single message** by sending the header `SAP_MessageProcessingLogLevel` with **None, Info or Debug** (Trace cannot be set this way) — handy for resending one message with more detail.

## 3. How Long Are Message Logs Kept?

> **Question:** till what time are message logs persisted?
> **Answer:** CPI will **not** persist message logs forever — it stores them for **30 days**. After 30 days the logs get deleted.

**Scenario:** if you want to see the log of **1 month back**, you can; but **more than a month** back you cannot.

**If you need longer retention:** you can **push the logs to an external logging system** — e.g. **Splunk, Loggly, Datadog, Elasticsearch**, etc.

> **Added context:** class named the tools but not the mechanism. Common approaches are custom logging from the iFlow (Groovy script writing to the MPL, or an HTTP call to the logging tool) and reading logs through the **Message Processing Logs OData API**. These are good topics to ask your trainer about in a later class.

## Part B — Data Formats & Converters

## 4. Data Formats in CPI

**What are all the data formats in CPI? ① XML ② JSON ③ CSV**

**Question:** what if you receive data as **JSON** from the input and need to send it to the target system in another format? **What will you do? → XML → JSON, or JSON → XML, and vice versa — at this time we use Converters.**

> **This answers the "CPI supports only XML" point flagged on Days 7, 9 and 12:** CPI handles XML, JSON and CSV — and the Converter steps are exactly how data moves between them.

## 5. The Six Converters (Palette → Transformation → Converter)

| # | Converter |
|---|---|
| ① | CSV to XML Converter |
| ② | EDI to XML Converter |
| ③ | JSON to XML Converter |
| ④ | XML to CSV Converter |
| ⑤ | XML to EDI Converter |
| ⑥ | XML to JSON Converter |

Notice the pattern: **every converter has XML on one side** — that's why XML is CPI's "pivot" format (Day 12).

## 6. XML vs. JSON — Root Node

| XML | JSON |
|---|---|
| **Root node is mandatory** (only once — 1:1) | Root is **not mandatory** |

**Why?** Suppose we send **multiple records**: in XML we cannot tell that we received multiple records **unless we have a root node** wrapping them. JSON can simply start with an array `[ ... ]` of records, so no root is needed.

## 7. XML to JSON Converter — The Options

### ① Suppress JSON Root Element

**We select this when we don't need the root tag in the JSON data output.**

### ② Use Namespace Mapping

Use Namespace Mapping if we want to **add the namespace prefix** of the incoming **XML input payload** into the output JSON data — then we **check** it. **Most of the time we uncheck this.**

### ③ JSON Prefix Separator

Options: **Colon ( : )**, Comma ( , ), Dot ( . ), Pipe ( | ), Semicolon ( ; ), Space. **We always use Colon.**

> **Verified:** this setting only applies when Namespace Mapping is selected — it separates the namespace prefix from the element name (e.g., `abc:Name`). So when Namespace Mapping is unchecked (the usual case), the separator doesn't matter.

### ④ JSON Output Encoding

**UTF-8** ✓

### ⑤ Streaming — Controlling Arrays

If we want to pass the **[ ] array** explicitly — for example when we **get a single record as input** but the target still expects an array — **we go to Streaming.**

**Streaming ✓ → "Convert XML elements to JSON Array":**

- **All** — every element becomes a JSON array
- **Specified ones** — only the elements whose absolute path you list (e.g., `/root/Emp`) become arrays

### Worked Example (using your Day 9 student data)

Input XML with **one** record:

```
<root>
  <Emp><Name>Bharath</Name><RollNo>1</RollNo></Emp>
</root>
```

| Converter Settings | JSON Output (illustrative) |
|---|---|
| Default | `{"root":{"Emp":{"Name":"Bharath","RollNo":"1"}}}` |
| Suppress JSON Root Element ✓ | `{"Emp":{"Name":"Bharath","RollNo":"1"}}` |

| | |
|---|---|
| Suppress Root ✓ + Streaming ✓ + Specified: `/root/Emp` | `{"Emp":[{"Name":"Bharath","RollNo":"1"}]}` |

> **Why streaming matters for arrays (verified):** without streaming, the converter looks at the whole XML — one `Emp` becomes an object, several become an array, so the output shape changes with the data. With streaming, the converter can't look ahead, so you declare in advance which elements are arrays. **Caution:** if an element can repeat but you didn't list it, you can get invalid JSON (duplicate keys) — when unsure, use "All" or list every repeating element.
>
> **Also good to know:** the converter turns **all values into JSON strings** (no numbers/booleans), and mixed-content XML isn't supported.

## 8. Quick Interview Answers

| Question | Short Answer |
|---|---|
| What are the log levels in CPI? | None, Error, Info (default), Debug, Trace — Trace shows content, expires in 10 min, payload kept 1 hour |
| Difference between Debug and Trace? | Debug shows the steps executed; Trace also shows the payload, headers and properties at each step |
| Which level for production? | Info (SAP's recommendation); Trace/Debug only in dev/test |
| How long are MPLs retained? | 30 days; push to an external logging tool for longer |
| How do you send a single record as a JSON array? | XML to JSON Converter → Streaming → Specified ones → give the element's absolute path |
| Why is the root node mandatory in XML but not in JSON? | XML needs one root to wrap multiple records; JSON can start with an array |